❹ Closes the connection, coordinated with ActorSystem termination

❺ Creates a projection

To complete the Kafka projection, let's now look at the handler. This handler does not connect to a database and therefore does not need to extend a `JdbcHandler` to process the message.

To process the event, create a `ProducerRecord`—the Kafka class you learned about in chapter 14. You instantiate this class with a subject, a key, and a value for each market event and send it using the `sendProducer` from the `MarketProjection`:

```
class MarketProjectionHandler ... {
  ...

  override def process(
      envelope: EventEnvelope[Market.Event]): Future[Done] = {
    log.debug(
      s"processing market event [$envelope] to topic [$topic]}")

    val event = envelope.event
    val serializedEvent = serialize(event)   ❶
    if (!serializedEvent.isEmpty) {
      val record =
        new ProducerRecord(topic, event.marketId, serializedEvent)
      producer.send(record).map { _ =>
        log.debug(s"published event [$event] to topic [$topic]}")
        Done                                 ❷
      }
    } else {
      Future.successful(Done)                ❷
    }
  ...
}
```

❶ Creates a record to push to Kafka

❷ Returns Done whether it pushes to Kafka or not

The event is serialized according to the producer settings. The key is `String`, and the value is an `Array[Byte]`. To serialize the